

Flutter

1. What is Flutter?

Flutter is a **free tool made by Google** used to **create mobile, web, and desktop apps** using a **single code**.

You write code once, and the app can run on **Android and iOS**.

In simple words:

Flutter helps developers make apps faster with one codebase.

2. History of Flutter

- Flutter was created by **Google**.
 - First introduced in **2017**.
 - Stable version released in **2018**.
 - It uses the **Dart programming language**.
-

3. Why Use Flutter? (Simple Reasons)

1. **Single Code**
 - One code works for Android and iOS.
 2. **Fast Development**
 - Hot Reload lets you see changes instantly.
 3. **Beautiful UI**
 - Easy to design attractive screens.
 4. **Free and Open Source**
 - Anyone can use it without cost.
 5. **High Performance**
 - Apps run smoothly like native apps.
-

4. Where Flutter is Used

- Mobile apps
 - Web apps
 - Desktop apps
 - UI prototypes
-

5. Advantages of Flutter

- Saves time
- Easy UI design
- Large community
- Good documentation

Step to Create Flutter Project

1. Open **Visual Studio Code**
→ Click **File > New Window**
2. Go to **View**
3. Select **Command Palette**
4. Click **Flutter: New Project**
5. Select **Application**
6. Select the folder where you want to store the project
7. Enter the project name
8. Use **lowercase letters only**
9. Example: my_first_app
10. Wait until the project is created successfully

1 Stateless Widget & Stateful Widget

➤ Stateless Widget

Definition

A **StatelessWidget** is a widget that **does not change its state** during runtime. Once it is built, its UI remains the same.

✓ Used when:

- UI is fixed
- No data change
- No user interaction affecting UI

Key Points

- Immutable (cannot change)
- build() method is called once
- Faster performance

Syntax

```
class MyWidget extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Widget;  
  }  
}
```

➤ Stateful Widget

Definition

A **StatefulWidget** is a widget that **can change its UI during runtime**.

✓ Used when:

- UI updates dynamically
 - Button clicks
 - Counters, forms, animations
-

Key Points

- Mutable
- Uses setState()
- Has two classes:
 - Widget class
 - State class

Syntax

```
class MyWidget extends StatefulWidget {  
  @override  
  _MyWidgetState createState() => _MyWidgetState();  
}  
  
class _MyWidgetState extends State<MyWidget> {  
  @override  
  Widget build(BuildContext context) {  
    return Widget;  
  }  
}
```

Example 1: Counter

```
class Counter extends StatefulWidget {  
  @override  
  _CounterState createState() => _CounterState();  
}  
  
class _CounterState extends State<Counter> {  
  int count = 0;  
  
  @override  
  Widget build(BuildContext context) {  
    return Column(  
      children: [  
        Text("Count: $count"),  
        ElevatedButton(  
          onPressed: () {  
            setState() {  
              count++;  
            }  
          }  
        ),  
        child: Text("Increment"),  
      ],  
    );  
  }  
}
```

2 Scaffold

Definition

Scaffold provides the **basic layout structure** of a Flutter app.

It supports:

- AppBar
- Body
- Drawer

- FloatingActionButton
- BottomNavigationBar
-

Syntax

```
Scaffold(  
  appBar: AppBar(),  
  body: Widget,  
)
```

Example 1: Simple Scaffold

```
Scaffold(  
  appBar: AppBar(title: Text("Home")),  
  body: Center(child: Text("Welcome")),  
);
```

3 AppBar

Definition

AppBar is the **top bar** of the app.

Used for:

- Title
- Actions
- Navigation

Syntax

```
AppBar(  
  title: Text("Title"),  
)
```

Example 1: Simple AppBar

```
AppBar(  
  title: Text("Flutter App"),  
);
```

Example 2: AppBar with Actions

```
AppBar(  
  title: Text("Home"),  
  actions: [  
    IconButton(  
      icon: Icon(Icons.search),
```

```
        onPressed: () {},
      ),
    ],
  );
```

Example 3: Colored AppBar

```
AppBar(
  title: Text("Custom"),
  backgroundColor: Colors.green,
);
```

4 Text Widget

Definition

Displays **text on the screen**.

Syntax

```
Text("Hello")
```

Example 1: Simple Text

```
Text("Flutter")
```

Example 2: Styled Text

```
Text(
  "Flutter UI",
  style: TextStyle(
    fontSize: 20,
    fontWeight: FontWeight.bold,
  ),
);
```

Example 3: Centered Colored Text

```
Text(
  "Welcome",
  style: TextStyle(color: Colors.blue),
  textAlign: TextAlign.center,
);
```

5 Color

Definition

Colors define the **visual appearance**.

Types of Colors

1. Material Colors

```
Colors.red  
Colors.blue
```

2. Custom Color

```
Color(0xFF00FF00)
```

3. Opacity Color

```
Colors.black.withOpacity(0.5)
```

Example 1

```
Container(color: Colors.orange)
```

Example 2

```
Text("Text", style: TextStyle(color: Color(0xFF2196F3)))
```

6 Center

Definition

Centers its child **both horizontally and vertically**.

Syntax

```
Center(child: Widget)
```

Example 1

```
Center(child: Text("Hello"))
```

Example 2

```
Center(  
  child: Container(  
    height: 100,  
    width: 100,  
    color: Colors.red,  
  ),  
);
```

7 SizedBox

Definition

Adds **fixed space or size**.

Syntax

```
SizedBox(height: 20)
```

Example 1: Space

```
SizedBox(height: 20)
```

Example 2: Sized Widget

```
SizedBox(  
  height: 100,  
  width: 100,  
  child: Container(color: Colors.blue),  
);
```

Example 3: Horizontal Space

```
SizedBox(width: 15)
```

Example 4: Simple Space Between Widgets

```
Column(  
  children: [  
    Text("First Line"),  
    SizedBox(height: 30), // Adds 30px vertical space  
    Text("Second Line"),  
  ],  
);
```

8 Container

Definition

A **box model widget** used for layout, decoration, margin, padding.

Syntax

```
Container(  
  height: 100,  
  width: 100,  
  color: Colors.red,
```



```
)
```

Example 1: Simple Container

```
Container(  
  height: 100,  
  width: 100,  
  color: Colors.green,  
);
```

Example 2: Padding & Margin

```
Container(  
  margin: EdgeInsets.all(10),  
  padding: EdgeInsets.all(15),  
  color: Colors.blue,  
  child: Text("Box"),  
);
```

Example 3: Decoration

```
Container(  
  height: 100,  
  width: 100,  
  decoration: BoxDecoration(  
    color: Colors.orange,  
    borderRadius: BorderRadius.circular(10),  
  ),  
);
```

Example 4: Container with Size, Color & Padding

```
Container(  
  width: 150,  
  height: 100,  
  padding: EdgeInsets.all(16),  
  color: Colors.blue,  
  child: Text(  
    'Blue Box',  
    style: TextStyle(color: Colors.white),  
  ),  
)
```

Example 5: Container with Margin, Border & Rounded Corners

```
Container(  
  margin: EdgeInsets.all(20),  
  padding: EdgeInsets.all(14),  
  decoration: BoxDecoration(  
    color: Colors.green,  
    border: Border.all(width: 3, color: Colors.black),  
    borderRadius: BorderRadius.circular(12),  
  ),  
  child: Text('Styled Container'),  
);
```

)

Simple Flutter Code

You can achieve that with the `borderRadius` property inside `BoxDecoration`:

```
Scaffold(  
  body: Center(  
    child: Container(  
      height: 200,  
      width: 300,  
      decoration: BoxDecoration(  
        color: Colors.blue,  
        borderRadius: BorderRadius.only(  
          topLeft: Radius.circular(40),  
          bottomLeft: Radius.circular(0),  
          topRight: Radius.circular(0),  
          bottomRight: Radius.circular(40),  
        ),  
        boxShadow: [  
          BoxShadow(  
            color: const Color.fromARGB(255, 212, 0, 0).withAlpha(50), // shadow color  
            blurRadius: 12, // softness  
            spreadRadius: 2, // size  
            offset: Offset(4, 6), // position (x, y)  
          ),  
        ],  
      ),  
    child: const Center(  
      child: Text(  
        "Custom Shape",  
        style: TextStyle(  
          color: Colors.white,  
          fontSize: 18,  
          fontWeight: FontWeight.w600,  
        ),  
      ),  
    ),  
  ),  
);
```

Basic Text

```
Text('Hello Flutter')
```

2 Styled Text

```
Text(  
  'Welcome',  
  style: TextStyle(  
    fontSize: 20,
```

```
fontWeight: FontWeight.bold,  
color: Colors.blue,  
),  
)
```

3 Text Alignment

```
Text(  
  'Center Text',  
  textAlign: TextAlign.center,  
)
```

4 Max Lines & Overflow

```
Text(  
  'This is a very long text that may overflow',  
  maxLines: 2,  
  overflow: TextOverflow.ellipsis,  
)
```

5 Custom Font (from pubspec.yaml)

```
Text(  
  'Custom Font Text',  
  style: TextStyle(  
    fontFamily: 'Poppins',  
    fontSize: 18,  
  )),  
)
```

6 Rich Text (Multiple Styles in One Line)

```
RichText(  
  text: TextSpan(  
    text: 'Hello ',  
    style: TextStyle(color: Colors.black, fontSize: 16),  
    children: [  
      TextSpan(  
        text: 'Flutter',  
        style: TextStyle(  
          color: Colors.blue,  
          fontWeight: FontWeight.bold,  
        )),  
    ],  
  ),  
)
```

7 Gradient Text

```
ShaderMask(  
  shaderCallback: (bounds) => LinearGradient(  
    colors: [Colors.red, Colors.orange],  
  ).createShader(bounds),  
  child: Text(  
    'Gradient Text',  
    style: TextStyle(  
      fontSize: 24,  
      color: Colors.white,  
      fontWeight: FontWeight.bold,  
    ),  
  ),  
)
```

8 Text with Shadow

```
Text(  
  'Shadow Text',  
  style: TextStyle(  
    fontSize: 22,  
    shadows: [  
      Shadow(  
        blurRadius: 5,  
        color: Colors.black45,  
        offset: Offset(2, 2),  
      ),  
    ],  
  ),  
)
```

9 Selectable Text

```
SelectableText('You can select this text')
```

Basic Syntax

```
Row(  
  children: [  
    Widget1(),  
    Widget2(),  
    Widget3(),  
  ],  
)
```

What is `Row` in Flutter?

`Row` is a **layout widget** that arranges its children **horizontally** (left → right).

- Think of it like a **horizontal line** where widgets sit next to each other.

□ Basic Syntax

```
Row(  
  children: [  
    Widget1(),  
    Widget2(),  
    Widget3(),  
  ],  
)
```

□ Example 1: Simple Row (Text Widgets)

This is the most basic use of Row.

```
Row(  
  children: const [  
    Text("Name"),  
    SizedBox(width: 10),  
    Text(":"),  
    SizedBox(width: 10),  
    Text("Dhanashree"),  
  ],  
)
```

□ Output

Name : Dhanashree in a single horizontal line.

□ Example 2: Row with `MainAxisAlignment`

This controls **horizontal spacing**.

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceBetween,  
  children: const [  
    Icon(Icons.home),  
    Icon(Icons.search),  
    Icon(Icons.settings),  
  ],  
)
```

□ What happens?

- Icons are spaced **evenly across the screen**
- Very common in **app bars, footers, menus**

Other useful options:

- `MainAxisAlignment.start`
- `MainAxisAlignment.center`
- `MainAxisAlignment.end`
- `MainAxisAlignment.spaceAround`
- `MainAxisAlignment.spaceEvenly`

□ **Example 3: Row with Expanded (Very Important □)**

Used to **share available width**.

```
Row(  
  children: [  
    Expanded(  
      flex: 2,  
      child: Container(  
        height: 50,  
        color: Colors.blue,  
        child: const Center(child: Text("Left")),  
      ),  
    ),  
    Expanded(  
      flex: 1,  
      child: Container(  
        height: 50,  
        color: Colors.green,  
        child: const Center(child: Text("Right")),  
      ),  
    ),  
  ],  
)
```

□ **Explanation**

- Total space = 3 parts
- Blue gets **2 parts**
- Green gets **1 part**
- Perfect for **forms, dashboards, split layouts**

□ **Bonus: Row with CrossAxisAlignment**

Controls **vertical alignment**.

```
Row(  
  crossAxisAlignment: CrossAxisAlignment.center,  
  children: const [  
    Icon(Icons.person, size: 40),  
    SizedBox(width: 10),  
    Text("Profile", style: TextStyle(fontSize: 20)),  
  ],  
)
```

```
],  
)
```

☐ Common Mistake (Overflow Error)

☐ This causes overflow:

```
Row(  
  children: [  
    Text("Very very very long text"),  
    Text("More text"),  
  ],  
)
```

☐ Fix using Expanded:

```
Row(  
  children: const [  
    Expanded(child: Text("Very very very long text")),  
    Text("More"),  
  ],  
)
```

What is `Column` in Flutter?

`Column` is a **layout widget** that arranges its **children vertically** (top → bottom).

☐ It's commonly used for:

- Forms
- Profile screens
- Login pages
- Cards & sections

☐ Basic Syntax

```
Column(  
  children: [  
    Widget1(),  
    Widget2(),  
    Widget3(),  
  ],  
)
```

☐ Example 1: Simple Vertical Text Layout

Use case: Display texts one below another.

```
Column(  
  children: const [  
    Text(  
      'Flutter',  
      style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),  
    ),  
    Text('UI Toolkit'),  
    Text('By Google'),  
  ],  
)
```

□ **Output:**

```
Flutter  
UI Toolkit  
By Google
```

□ **Example 2: Column with Alignment & Spacing**

Use case: Center content vertically and horizontally.

```
Column(  
  mainAxisAlignment: MainAxisAlignment.center, // vertical  
  crossAxisAlignment: CrossAxisAlignment.center, // horizontal  
  children: const [  
    Icon(Icons.flutter_dash, size: 60, color: Colors.blue),  
    SizedBox(height: 10),  
    Text(  
      'Welcome to Flutter',  
      style: TextStyle(fontSize: 18),  
    ),  
  ],  
)
```

□ **Key Properties:**

- `mainAxisAlignment` → vertical alignment
 - `crossAxisAlignment` → horizontal alignment
 - `SizedBox` → spacing between widgets
-

□ **Example 3: Column Inside Container (Card-like UI)**

Use case: Profile / Info card.

```
Container(  
  padding: const EdgeInsets.all(16),  
  decoration: BoxDecoration(  
    color: Colors.white,  
    borderRadius: BorderRadius.circular(12),  
  ),  
)
```



```

boxShadow: [
  BoxShadow(
    color: Colors.black.withOpacity(0.1),
    blurRadius: 10,
    offset: const Offset(0, 4),
  ),
],
),
child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: const [
    Text(
      'Dhanashree Wagh',
      style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
    ),
    SizedBox(height: 8),
    Text('Flutter Developer'),
    SizedBox(height: 4),
    Text('UI/UX Designer'),
  ],
),
)

```

- ❑ Very common in dashboards, lists & cards.

❑ Example 4: Column with Expanded Widgets

Use case: Divide vertical space equally.

```

Column(
  children: [
    Expanded(
      child: Container(color: Colors.red),
    ),
    Expanded(
      child: Container(color: Colors.green),
    ),
    Expanded(
      child: Container(color: Colors.blue),
    ),
  ],
)

```

- ❑ Each container takes **equal height**.

❑ Important Column Properties (Quick Table)

Property	Purpose
children	Widgets inside Column
mainAxisAlignment	Vertical alignment

Property	Purpose
crossAxisAlignment	Horizontal alignment
mainAxisSize	Min or Max height
Expanded	Flexible height

❑ Common Mistake

❑ Overflow error

```
Column(
  children: [
    // too many widgets without scroll
  ],
)
```

❑ Fix using SingleChildScrollView

```
SingleChildScrollView(
  child: Column(
    children: [],
  ),
)
```

```
import 'package:flutter/material.dart';
```

```
class RowColumnExample extends StatelessWidget {
```

```
  const RowColumnExample({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return Scaffold(
```

```
      appBar: AppBar(
```

```
        title: const Text("Row & Column Example"),
```

```
      ),
```

```
      body: Center(
```

```
        child: Container(
```

```
          width: 300,
```

```
padding: const EdgeInsets.all(16),

decoration: BoxDecoration(

  color: Colors.blue.shade50,

  borderRadius: BorderRadius.circular(16),

  boxShadow: [

    BoxShadow(

      color: Colors.black.withOpacity(0.15),

      blurRadius: 8,

      offset: const Offset(0, 4),

    ),

  ],

),

// COLUMN → vertical layout

child: Column(

  mainAxisAlignment: MainAxisAlignment.min,

  crossAxisAlignment: CrossAxisAlignment.start,

  children: [

    // ROW → horizontal layout

    Row(

      children: const [

        Icon(

          Icons.flutter_dash,

          color: Colors.blue,

          size: 30,

        ),
```

```
      SizedBox(width: 8),

      Text(
        "Flutter UI",
        style: TextStyle(
          fontSize: 20,
          fontWeight: FontWeight.bold,
        ),
      ),
    ],
  ),
```

```
const SizedBox(height: 8),
```

```
const Text(
  "Simple Row and Column layout",
  style: TextStyle(
    fontSize: 14,
    color: Colors.black54,
  ),
),
```

```
const SizedBox(height: 12),
```

```
// Another ROW inside COLUMN
```

```
Row(
  children: const [
    Icon(
```

```
Icons.star,  
  
color: Colors.orange,  
  
size: 20,  
  
)  
  
SizedBox(width: 6),  
  
Text(  
  
  "Rating: 4.8",  
  
  style: TextStyle(  
  
    fontSize: 16,  
  
  ),  
  
)  
  
],  
  
),  
  
],  
  
)  
  
)  
  
)  
  
)  
  
);  
  
}  
  
}
```

Lists & Builder Widgets in Flutter

In Flutter, **list-based widgets** are used to display **multiple items** in a scrollable format, such as contacts, products, messages, or cards.

Flutter mainly provides:

- ListView
- GridView
- Their builder versions:
 - ListView.builder
 - GridView.builder

1 ListView

Definition

ListView is a scrollable widget that displays its children **one after another in a vertical or horizontal direction**.

When to use

- When you have **small or fixed number of widgets**
- When items are **known in advance**
- Simple UI without performance concerns

Syntax

```
ListView(  
  children: [  
    Widget1,  
    Widget2,  
    Widget3,  
  ],  
)
```

Example: Simple ListView with Container, Text & Icon

```
ListView(  
  padding: const EdgeInsets.all(16),  
  children: [  
    Container(  
      height: 70,  
      margin: const EdgeInsets.only(bottom: 10),  
      decoration: BoxDecoration(  
        color: Colors.blue.shade100,  
        borderRadius: BorderRadius.circular(10),  
      ),  
      child: const Row(  
        children: [  
          SizedBox(width: 10),  
          Icon(Icons.person, size: 30),  
          SizedBox(width: 10),  
          Text(  
            'John Doe',  
            style: TextStyle(fontSize: 18),  
          ),  
        ],  
      ),  
    ),  
    Container(  
      height: 70,  
      decoration: BoxDecoration(  
        color: Colors.green.shade100,  
        borderRadius: BorderRadius.circular(10),  
      ),  
    ),  
  ],  
)
```

```
child: const Row(  
  children: [  
    SizedBox(width: 10),  
    Icon(Icons.person, size: 30),  
    SizedBox(width: 10),  
    Text(  
      'Jane Smith',  
      style: TextStyle(fontSize: 18),  
    ),  
  ],  
,  
,  
,  
),  
],  
)
```

2 GridView

Definition

GridView displays items in a **two-dimensional grid layout** (rows & columns).

When to use

- Dashboards
- Product listings
- Photo galleries
- Menu screens

Syntax

```
GridView.count(  
  crossAxisCount: 2,  
  children: [],  
)
```

Example: Simple GridView

```
GridView.count(  
  crossAxisCount: 2,  
  padding: const EdgeInsets.all(12),  
  crossAxisSpacing: 10,  
  mainAxisSpacing: 10,  
  children: List.generate(4, (index) {  
    return Container(  
      decoration: BoxDecoration(  
        color: Colors.orange.shade100,  
        borderRadius: BorderRadius.circular(12),  
      ),  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: [  
          Icon(Icons.dashboard, size: 40),  
          const SizedBox(height: 8),  
          Text(  

```

```

        'Item ${index + 1}',
        style: const TextStyle(fontSize: 16),
      ),
    ],
  ),
);
}),
)

```

3 ListView.builder

Definition

ListView.builder creates list items **dynamically and lazily** (only visible items are built).

Why use it?

- Best for **large or dynamic data**
- Better **performance**
- Data comes from API / database / Firebase / Supabase

Syntax

```

ListView.builder(
  itemCount: itemCount,
  itemBuilder: (context, index) {
    return Widget;
  },
)

```

Example: ListView.builder with Custom Card

```

ListView.builder(
  padding: const EdgeInsets.all(16),
  itemCount: 5,
  itemBuilder: (context, index) {
    return Container(
      margin: const EdgeInsets.only(bottom: 12),
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        gradient: LinearGradient(
          colors: [Colors.blue.shade200, Colors.blue.shade50],
        ),
      ),
      borderRadius: BorderRadius.circular(12),
      boxShadow: [
        BoxShadow(
          color: Colors.black.withOpacity(0.1),
          blurRadius: 6,
          offset: const Offset(0, 3),
        ),
      ],
    ),
  ),
  child: Row(
    children: [

```



```

const Icon(Icons.account_circle, size: 40),
const SizedBox(width: 12),
Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text(
      'User ${index + 1}',
      style: const TextStyle(
        fontSize: 18,
        fontWeight: FontWeight.bold,
      ),
    ),
    const Text('Flutter Developer'),
  ],
),
],
),
);
},
)

```

4 GridView.builder

Definition

GridView.builder dynamically builds grid items **only when needed**, similar to ListView.builder.

When to use

- Large number of grid items
- API-based dashboards
- Performance-optimized layouts

Syntax

```

GridView.builder(
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
  ),
  itemBuilder: (context, index) {},
)

```

Example: GridView.builder with Custom Cards

```

GridView.builder(
  padding: const EdgeInsets.all(12),
  itemCount: 6,
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
    mainAxisSpacing: 12,
    crossAxisSpacing: 12,
    childAspectRatio: 1,
  ),
  itemBuilder: (context, index) {

```

```

return Container(
  decoration: BoxDecoration(
    color: Colors.purple.shade100,
    borderRadius: BorderRadius.circular(16),
    boxShadow: [
      BoxShadow(
        color: Colors.black.withOpacity(0.15),
        blurRadius: 8,
      ),
    ],
  ),
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      const Icon(Icons.star, size: 40),
      const SizedBox(height: 8),
      Text(
        'Card ${index + 1}',
        style: const TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.w600,
        ),
      ),
    ],
  ),
);

```

5 Custom Cards in Flutter

Definition

A **custom card** is a reusable UI component built using Container, BoxDecoration, Row, Column, etc., instead of the default Card widget.

Why custom cards?

- Full design control
- Gradients & shadows
- Consistent UI across app
- Matches Figma/UI designs (useful for you as a UI/UX dev )

Example: Reusable Custom Card Widget

```

class CustomInfoCard extends StatelessWidget {
  final IconData icon;
  final String title;
  final String subtitle;

  const CustomInfoCard({
    super.key,
    required this.icon,
    required this.title,

```

```
        required this.subtitle,
    });

    @override
    Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        color: Colors.teal.shade100,
        borderRadius: BorderRadius.circular(14),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.1),
            blurRadius: 6,
          ),
        ],
      ),
      child: Row(
        children: [
          Icon(icon, size: 36),
          const SizedBox(width: 12),
          Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                title,
                style: const TextStyle(
                  fontSize: 18,
                  fontWeight: FontWeight.bold,
                ),
              ),
              Text(subtitle),
            ],
          ),
        ],
      ),
    );
  }
}
```

❑ Summary Table

Widget	Use Case	Performance
ListView	Small static lists	Medium
GridView	Static grid layouts	Medium
ListView.builder	Large / dynamic lists	High
GridView.builder	Large / dynamic grids	High
Custom Cards	UI customization	Best

Types of Images in Flutter

Flutter supports mainly **4 ways** to show images:

1. **Asset Image** (local image)
 2. **Network Image** (from internet)
 3. **File Image** (from device storage)
 4. **Memory Image** (bytes)
-

❑ 1 Asset Image (Most Common)

Step 1: Add image to project

```
assets/images/logo.png
```

Step 2: Update pubspec.yaml

```
flutter:  
  assets:  
    - assets/images/logo.png
```

Step 3: Use in UI

```
Image.asset(  
  'assets/images/logo.png',  
  height: 120,  
  width: 120,  
  fit: BoxFit.cover,  
)
```

❑ 2 Network Image (From URL)

```
Image.network(  
  'https://example.com/image.jpg',  
  height: 150,  
  width: 150,  
  fit: BoxFit.cover,  
)
```

With loading indicator

```
Image.network(  
  'https://example.com/image.jpg',  
  loadingBuilder: (context, child, progress) {  
    if (progress == null) return child;  
    return CircularProgressIndicator();  
  },  
)
```

❑ 3 Image inside Container (Rounded / Styled)

```
Container(  
  height: 150,  
  width: 150,  
  decoration: BoxDecoration(  
    borderRadius: BorderRadius.circular(12),  
    image: DecorationImage(  
      image: AssetImage('assets/images/logo.png'),  
      fit: BoxFit.cover,  
    ),  
  ),  
)
```

❑ Best for **cards, banners, profile photos**

❑ 4 Circular Image (Profile Avatar)

```
CircleAvatar(  
  radius: 40,  
  backgroundImage: AssetImage('assets/images/profile.png'),  
)
```

OR (Network)

```
CircleAvatar(  
  radius: 40,  
  backgroundImage: NetworkImage(  
    'https://example.com/profile.jpg',  
  ),  
)
```

❑ 5 Image with Icon Overlay (Camera / Edit)

(You used this in visitor registration ❑)

```
Stack(  
  alignment: Alignment.bottomRight,  
  children: [  
    CircleAvatar(  
      radius: 45,  
      backgroundImage: AssetImage('assets/images/profile.png'),  
    ),  
    CircleAvatar(  
      radius: 14,  
      backgroundColor: Colors.blue,  
      child: Icon(Icons.camera_alt, size: 16, color: Colors.white),  
    ),  
  ],  
)
```

❑ 6❑ Image in Card

```
Card(  
  elevation: 6,  
  shape: RoundedRectangleBorder(  
    borderRadius: BorderRadius.circular(12),  
  ),  
  child: Column(  
    children: [  
      ClipRRect(  
        borderRadius: BorderRadius.vertical(top: Radius.circular(12)),  
        child: Image.asset(  
          'assets/images/banner.png',  
          height: 120,  
          width: double.infinity,  
          fit: BoxFit.cover,  
        ),  
      ),  
      Padding(  
        padding: EdgeInsets.all(12),  
        child: Text("Card with Image"),  
      ),  
    ],  
  ),  
)
```

What is a Card in Flutter?

A **Card** is a Material Design widget used to display related information in a **container with elevation, rounded corners, and shadow**.

❑ Common uses:

- Product cards
- Profile cards
- List items
- Dashboards

❑ Basic Card Structure

```
Card(  
  child: Widget,  
)
```

❑ Simple Card Example

```
Card(  
  elevation: 5,  
  margin: const EdgeInsets.all(10),  
  child: Padding(  

```

```
padding: const EdgeInsets.all(16),
child: Text(
  "This is a simple Card",
  style: TextStyle(fontSize: 16),
),
),
)
```

❑ Card with Icon + Text (Very Common UI)

```
Card(
  elevation: 6,
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(12),
  ),
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Row(
      children: [
        Icon(Icons.person, size: 40, color: Colors.blue),
        SizedBox(width: 12),
        Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              "John Doe",
              style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
            ),
            Text("Flutter Developer"),
          ],
        ),
      ],
    ),
  ),
)
```

❑ Card with ListView.builder (Real-World Use)

```
ListView.builder(
  itemCount: 5,
  itemBuilder: (context, index) {
    return Card(
      elevation: 4,
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      child: ListTile(
        leading: Icon(Icons.event),
        title: Text("Event ${index + 1}"),
        subtitle: Text("Event description"),
        trailing: Icon(Icons.arrow_forward_ios, size: 16),
      ),
    );
  },
)
```

❑ Card with Custom Design (Gradient + Shadow)

Since you like **custom UI**, this is important
(Using Container inside Card)

```
Card(  
  elevation: 8,  
  shape: RoundedRectangleBorder(  
    borderRadius: BorderRadius.circular(16),  
  ),  
  child: Container(  
    decoration: BoxDecoration(  
      borderRadius: BorderRadius.circular(16),  
      gradient: LinearGradient(  
        colors: [Colors.purple, Colors.blue],  
        begin: Alignment.topLeft,  
        end: Alignment.bottomRight,  
      ),  
    ),  
    padding: const EdgeInsets.all(16),  
    child: Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
        Text(  
          "Premium Plan",  
          style: TextStyle(  
            color: Colors.white,  
            fontSize: 18,  
            fontWeight: FontWeight.bold,  
          ),  
        ),  
        SizedBox(height: 8),  
        Text(  
          "₹999 / month",  
          style: TextStyle(color: Colors.white70),  
        ),  
      ],  
    ),  
  ),  
)
```

```
Card(  
  
  elevation: 0,  
  
  margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),  
  
  shape: RoundedRectangleBorder(  
  
    borderRadius: BorderRadius.circular(16),  
  
  ),  
  
  child: Container(  
  
    decoration: BoxDecoration(  

```



```
color: Colors.white,

borderRadius: BorderRadius.circular(16),

boxShadow: [

  BoxShadow(

    color: Colors.black.withOpacity(0.08),

    blurRadius: 12,

    offset: Offset(0, 6),

  ),

],

),

child: Column(

  crossAxisAlignment: CrossAxisAlignment.start,

  children: [

    ClipRRect(

      borderRadius: const BorderRadius.vertical(

        top: Radius.circular(16),

      ),

      child: Image.asset(

        'assets/images/sample.jpg',

        height: 140,

        width: double.infinity,

        fit: BoxFit.cover,

      ),

    ),

    Padding(

      padding: const EdgeInsets.all(12),

      child: Column(
```

```
crossAxisAlignment: CrossAxisAlignment.start,

children: const [

  Text(

    "Flutter UI Card",

    style: TextStyle(

      fontSize: 16,

      fontWeight: FontWeight.w600,

    ),

  ),

  SizedBox(height: 6),

  Text(

    "Image, text and shadow styled card layout",

    style: TextStyle(

      fontSize: 13,

      color: Colors.grey,

    ),

    maxLines: 2,

    overflow: TextOverflow.ellipsis,

  ),

],

),

),

],

),

),

)

class SimpleFormExample extends StatelessWidget {
```

```
final _formKey = GlobalKey<FormState>();

final TextEditingController nameController = TextEditingController();
```

```
@override
```

```
Widget build(BuildContext context) {

  return Scaffold(

    appBar: AppBar(title: const Text("Simple Form")),

    body: Padding(

      padding: const EdgeInsets.all(16),

      child: Form(

        key: _formKey,

        child: Column(

          children: [

            TextFormField(

              controller: nameController,

              decoration: const InputDecoration(

                labelText: "Name",

              ),

              validator: (value) {

                if (value == null || value.isEmpty) {

                  return "Enter name";

                }

                return null;

              },

            ),

            const SizedBox(height: 20),

            ElevatedButton(
```

```

        onPressed: () {

          if (_formKey.currentState!.validate()) {

            print("Name: ${nameController.text}");

          }

        },

        child: const Text("Submit"),

      ),

    ],

  ),

),

),

),

);

}

}

class OutlinedFormExample extends StatelessWidget {

  final TextEditingController emailController = TextEditingController();

  @override

  Widget build(BuildContext context) {

    return Scaffold(

      appBar: AppBar(title: const Text("Outlined Input")),

      body: Padding(

        padding: const EdgeInsets.all(16),

        child: TextFormField(

          controller: emailController,

          decoration: InputDecoration(

            labelText: "Email",

```

```

        hintText: "example@gmail.com",

        border: OutlineInputBorder(

          borderRadius: BorderRadius.circular(12),

        ),

        enabledBorder: OutlineInputBorder(

          borderRadius: BorderRadius.circular(12),

          borderSide: const BorderSide(color: Colors.grey),

        ),

        focusedBorder: OutlineInputBorder(

          borderRadius: BorderRadius.circular(12),

          borderSide: const BorderSide(color: Colors.blue, width: 2),

        ),

      ),

    ),

  ),

);

}

}

import 'package:flutter/material.dart';

class GradientFormValidation extends StatefulWidget {

  const GradientFormValidation({super.key});

  @override

  State<GradientFormValidation> createState() => _GradientFormValidationState();

}

class _GradientFormValidationState extends State<GradientFormValidation> {

```

```
final _formKey = GlobalKey<FormState>();
```

```
final TextEditingController emailController = TextEditingController();
```

```
final TextEditingController passwordController = TextEditingController();
```

```
Widget gradientInputField({  
  required String label,  
  required TextEditingController controller,  
  bool isPassword = false,  
  String? Function(String?)? validator,  
}) {  
  return Container(  
    margin: const EdgeInsets.only(bottom: 20),  
    padding: const EdgeInsets.all(2),  
    decoration: BoxDecoration(  
      borderRadius: BorderRadius.circular(14),  
      gradient: const LinearGradient(  
        colors: [Colors.purple, Colors.blue],  
      ),  
    ),  
    child: Container(  
      padding: const EdgeInsets.symmetric(horizontal: 14),  
      decoration: BoxDecoration(  
        color: Colors.white,  
        borderRadius: BorderRadius.circular(12),  
      ),  
      child: TextFormField(  

```



```
        return "Email is required";
    }

    if (!value.contains("@")) {
        return "Enter valid email";
    }

    return null;
},
),
gradientInputField(
    label: "Password",
    controller: passwordController,
    isPassword: true,
    validator: (value) {
        if (value == null || value.isEmpty) {
            return "Password is required";
        }

        if (value.length < 6) {
            return "Minimum 6 characters";
        }

        return null;
    },
),
const SizedBox(height: 10),
ElevatedButton(
    style: ElevatedButton.styleFrom(
        padding: const EdgeInsets.symmetric(
            horizontal: 40, vertical: 14),
```



```
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),
  onPressed: () {
    if (_formKey.currentState!.validate()) {
      debugPrint("Email: ${emailController.text}");
      debugPrint("Password: ${passwordController.text}");
    }
  },
  child: const Text("Submit"),
),
],
),
),
),
);
}
}
```

```
import 'package:flutter/material.dart';

import 'package:firebase_auth/firebase_auth.dart';

import 'package:cloud_firestore/cloud_firestore.dart';

class StudentRegisterScreen extends StatefulWidget {

  @override

  State<StudentRegisterScreen> createState() =>

    _StudentRegisterScreenState();

}

class _StudentRegisterScreenState extends State<StudentRegisterScreen> {

  final _formKey = GlobalKey<FormState>();

  final nameCtrl = TextEditingController();

  final emailCtrl = TextEditingController();

  final passCtrl = TextEditingController();

  final phoneCtrl = TextEditingController();

  final addressCtrl = TextEditingController();

  final classCtrl = TextEditingController();

  final courseCtrl = TextEditingController();
```

```
bool loading = false;
```

```
Future<void> registerStudent() async {
```

```
  if (!_formKey.currentState!.validate()) return;
```

```
  try {
```

```
    setState(() => loading = true);
```

```
    // Firebase Auth
```

```
    UserCredential user =
```

```
      await FirebaseAuth.instance.createUserWithEmailAndPassword(
```

```
        email: emailCtrl.text.trim(),
```

```
        password: passCtrl.text.trim(),
```

```
      );
```

```
    // Firestore
```

```
    await FirebaseFirestore.instance
```

```
      .collection('students')
```

```
      .doc(user.user!.uid)
```

```
      .set({
```

```
        "name": nameCtrl.text,
```

```
        "email": emailCtrl.text,
```

```
        "phone": phoneCtrl.text,
```

```
        "address": addressCtrl.text,
```

```
        "class": classCtrl.text,
```

```
        "course": courseCtrl.text,
```

```

        "created_at": Timestamp.now(),
    });

    ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text("Registration Successful")),
    );

    _formKey.currentState!.reset();
} catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(e.toString())),
    );
} finally {
    setState(() => loading = false);
}
}

```

```

Widget inputField(String label, TextEditingController ctrl,
    {bool isPassword = false}) {
    return Container(
        margin: const EdgeInsets.only(bottom: 16),
        decoration: BoxDecoration(
            gradient: const LinearGradient(
                colors: [Color(0xff6a11cb), Color(0xff2575fc)],
            ),
            borderRadius: BorderRadius.circular(12),
        ),
    ),

```

```
child: TextFormField(
  controller: ctrl,
  obscureText: isPassword,
  style: const TextStyle(color: Colors.white),
  decoration: InputDecoration(
    hintText: label,
    hintStyle: const TextStyle(color: Colors.white70),
    border: InputBorder.none,
    contentPadding: const EdgeInsets.all(16),
  ),
  validator: (value) =>
    value!.isEmpty ? "Enter $label" : null,
),
);
}
```

@override

```
Widget build(BuildContext context) {
  return Scaffold(
    body: Container(
      padding: const EdgeInsets.all(20),
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          colors: [Color(0xff1d2671), Color(0xffc33764)],
          begin: Alignment.topLeft,
          end: Alignment.bottomRight,
        ),
      ),
```

```
),  
  
child: Center(  
  
  child: SingleChildScrollView(  
  
    child: Form(  
  
      key: _formKey,  
  
      child: Column(  
  
        children: [  
  
          const Text(  
  
            "Student Registration",  
  
            style: TextStyle(  
  
              fontSize: 26,  
  
              fontWeight: FontWeight.bold,  
  
              color: Colors.white,  
  
            ),  
  
          ),  
  
          const SizedBox(height: 30),  
  
          inputField("Name", nameCtrl),  
  
          inputField("Email", emailCtrl),  
  
          inputField("Password", passCtrl, isPassword: true),  
  
          inputField("Phone No", phoneCtrl),  
  
          inputField("Address", addressCtrl),  
  
          inputField("Class", classCtrl),  
  
          inputField("Course", courseCtrl),  
  
          const SizedBox(height: 20),
```

```
GesturesDetector(
  onTap: loading ? null : registerStudent,
  child: Container(
    height: 50,
    width: double.infinity,
    alignment: Alignment.center,
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(12),
      color: Colors.white,
    ),
    child: loading
      ? const CircularProgressIndicator()
      : const Text(
          "Register",
          style: TextStyle(
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
  ),
),
],
),
),
),
),
),
),
),
),
```

```

    );
  }
}

import 'package:flutter/material.dart';

import 'package:firebase_auth/firebase_auth.dart';

import 'home_screen.dart';

class LoginScreen extends StatefulWidget {

  @override

  State<LoginScreen> createState() => _LoginScreenState();

}

class _LoginScreenState extends State<LoginScreen> {

  final emailCtrl = TextEditingController();

  final passCtrl = TextEditingController();

  final _formKey = GlobalKey<FormState>();

  bool loading = false;

  Future<void> loginUser() async {

    if (!_formKey.currentState!.validate()) return;

    try {

      setState(() => loading = true);

      await FirebaseAuth.instance.signInWithEmailAndPassword(

        email: emailCtrl.text.trim(),

        password: passCtrl.text.trim(),

      );

```



```
Navigator.pushReplacement(  
  context,  
  MaterialPageRoute(builder: (_) => HomeScreen()),  
);  
} catch (e) {  
  ScaffoldMessenger.of(context)  
    .showSnackBar(SnackBar(content: Text(e.toString())));  
} finally {  
  setState(() => loading = false);  
}  
}
```

```
Widget inputField(String hint, TextEditingController ctrl,  
  {bool isPassword = false}) {  
  return Container(  
    margin: const EdgeInsets.only(bottom: 16),  
    decoration: BoxDecoration(  
      gradient: const LinearGradient(  
        colors: [Color(0xff6a11cb), Color(0xff2575fc)],  
      ),  
      borderRadius: BorderRadius.circular(12),  
    ),  
    child: TextFormField(  
      controller: ctrl,  
      obscureText: isPassword,  
      style: const TextStyle(color: Colors.white),
```

```
decoration: InputDecoration(
  hintText: hint,
  hintStyle: const TextStyle(color: Colors.white70),
  border: InputBorder.none,
  contentPadding: const EdgeInsets.all(16),
),
validator: (v) => v!.isEmpty ? "Enter $hint" : null,
),
);
}
```

@override

```
Widget build(BuildContext context) {
  return Scaffold(
    body: Container(
      padding: const EdgeInsets.all(20),
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          colors: [Color(0xff141E30), Color(0xff243B55)],
        ),
      ),
      child: Center(
        child: Form(
          key: _formKey,
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
```

```
const Text(  
  "Student Login",  
  style: TextStyle(  
    color: Colors.white,  
    fontSize: 26,  
    fontWeight: FontWeight.bold),  
),  
const SizedBox(height: 30),  
  
inputField("Email", emailCtrl),  
inputField("Password", passCtrl, isPassword: true),  
  
const SizedBox(height: 20),  
GestureDetector(  
  onTap: loading ? null : loginUser,  
  child: Container(  
    height: 50,  
    width: double.infinity,  
    alignment: Alignment.center,  
    decoration: BoxDecoration(  
      color: Colors.white,  
      borderRadius: BorderRadius.circular(12)),  
    child: loading  
      ? const CircularProgressIndicator()  
      : const Text("Login",  
        style: TextStyle(  
          fontSize: 18,
```

```

        fontWeight: FontWeight.bold)),

    ),

  ),

  ],

),

),

),

),

);

}

}

import 'package:flutter/material.dart';

import 'package:cloud_firestore/cloud_firestore.dart';

import 'package:firebase_auth/firebase_auth.dart';

import 'login_screen.dart';

class HomeScreen extends StatelessWidget {

  void logout(BuildContext context) async {

    await FirebaseAuth.instance.signOut();

    Navigator.pushReplacement(

      context, MaterialPageRoute(builder: (_) => LoginScreen()));

  }

  @override

  Widget build(BuildContext context) {

    return Scaffold(

      appBar: AppBar(

        title: const Text("Students List"),

```

```
actions: [

  IconButton(

    icon: const Icon(Icons.logout),

    onPressed: () => logout(context),

  )

],

),

body: StreamBuilder<QuerySnapshot>(

  stream:

    FirebaseFirestore.instance.collection('students').snapshots(),

  builder: (context, snapshot) {

    if (!snapshot.hasData) {

      return const Center(child: CircularProgressIndicator());

    }

    final docs = snapshot.data!.docs;

    if (docs.isEmpty) {

      return const Center(child: Text("No Students Found"));

    }

    return ListView.builder(

      padding: const EdgeInsets.all(16),

      itemCount: docs.length,

      itemBuilder: (context, index) {

        final data = docs[index];
```

```
return Container(  
  margin: const EdgeInsets.only(bottom: 16),  
  decoration: BoxDecoration(  
    borderRadius: BorderRadius.circular(16),  
    gradient: const LinearGradient(  
      colors: [Color(0xff8360c3), Color(0xff2ebf91)],  
    ),  
  ),  
  child: Padding(  
    padding: const EdgeInsets.all(16),  
    child: Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
        Text(  
          data['name'],  
          style: const TextStyle(  
            fontSize: 20,  
            fontWeight: FontWeight.bold,  
            color: Colors.white),  
        ),  
        const SizedBox(height: 8),  
        Text("Email: ${data['email']}",  
          style: const TextStyle(color: Colors.white)),  
        Text("Phone: ${data['phone']}",  
          style: const TextStyle(color: Colors.white)),  
        Text("Class: ${data['class']}",  
          style: const TextStyle(color: Colors.white)),
```

```
        Text("Course: ${data['course']}",
            style: const TextStyle(color: Colors.white)),
        Text("Address: ${data['address']}",
            style: const TextStyle(color: Colors.white)),
    ],
),
),
);
},
);
},
),
);
}
}
```

Step to add Firebase to your project

- delete the google-services.json file
android/app/google-services.json
- delete file firebase.json
- now run the command flutter clean
- then run flutter pub get
- create a project in firebase
- Firebase
- Go to console
- Get started with new project
- Name the project
- Tick the checkbox
- Continue
- Continue
- Select India in state and tick the checkbox
- Create project
- Continue
- Build
- Firebase Database
- Create Database
- Next
- Create
- Now your database is create enter the table in it
- Windows power shell open as administrator
- set-ExecutionPolicy RemoteSigned -Scope CurrentUser
- Get-ExecutionPolicy
- Get-ExecutionPolicy –list
- Vscode
- New project

- Terminal
- npm install -g firebase-tools
- npm install -g npm@11.2.0 only if npm is already install
- dart pub global activate flutterfire_cli
- C:\Users\Server\AppData\Local\Pub\Cache\bin add path to environment variable
- now in vscode
- firebase login
- enter y
- then firebase cli select the account
- close the vscode and open once again and terminal
- dart pub global activate flutterfire_cli check no warning
- flutterfire configure
- select platform windows,web,android,ios
- y
- add dependency to pubspec.yaml file
- firebase_core: ^3.12.1
- firebase_database: ^11.3.4
- firebase_storage: ^12.4.4
- firebase_auth: ^5.5.1
- cloud_firestore: ^5.6.5
- Add below code to main file
- Future<void> main() async {
- WidgetsFlutterBinding.ensureInitialized();
- await Firebase.initializeApp(options:
- DefaultFirebaseOptions.currentPlatform);
- runApp(const MyApp());
- }

Note : First 4 step only when firebase already integrate

Summary Basic Notes

What is Flutter?

Flutter is an **open-source UI (User Interface) toolkit** developed by Google.

It is used to build:

- Android apps
- iOS apps
- Web apps
- Desktop apps (Windows, macOS, Linux)

Flutter allows developers to create **beautiful, fast, and natively compiled applications** using a single codebase.

It is:

- ☐ A UI toolkit
- ☐ Not an operating system
- ☐ Not a database
- ☐ Not just a programming language

Flutter works using **widgets** to build everything on the screen.

2 Programming Language Used in Flutter

Flutter apps are developed using **Dart**.

Why Dart?

- Easy to learn
- Object-Oriented
- Fast execution
- Supports Hot Reload
- Compiles to native machine code

Example:

```
void main() {  
  runApp(MyApp());  
}
```

3 Basic Structure of a Flutter App

The main file of a Flutter project is:

□ **main.dart**

This file contains:

- `main()` function
- `runApp()` method
- Root widget of the application

Entry Point:

```
void main() {  
  runApp(MyApp());  
}
```

The `main()` function is where execution begins.

4 Important Core Widgets

Flutter UI is made of widgets. Everything is a widget.

Scaffold

Provides the basic layout structure of an app.

It includes:

- AppBar
- Body
- Drawer
- Floating Action Button
- Bottom Navigation Bar

Example:

```
Scaffold(  
  appBar: AppBar(title: Text("My App")),  
  body: Center(child: Text("Hello")),  
);
```

Text

Used to display text on the screen.

```
Text("Hello Flutter")
```

Image

Used to display images.

```
Image.asset("assets/image.png")  
Image.network("https://example.com/image.jpg")
```

Icon

Used to display icons.

```
Icon(Icons.home)
```

5 Layout Widgets

Layout widgets help arrange UI elements.

Column (Vertical Layout)

Arranges widgets from top to bottom.

```
Column(  
  children: [  
    Text("One"),  
    Text("Two"),  
  ],  
)
```

Row (Horizontal Layout)

Arranges widgets from left to right.

```
Row(  
  children: [  
    Text("A"),  
    Text("B"),  
  ],  
)
```

```
],  
)
```

Stack (Overlay Widgets)

Places widgets on top of each other.

```
Stack(  
  children: [  
    Image.asset("background.png"),  
    Text("Overlay Text"),  
  ],  
)
```

Center

Centers its child widget.

```
Center(  
  child: Text("Centered"),  
)
```

Expanded

Expands a widget to fill available space inside Row or Column.

```
Row(  
  children: [  
    Expanded(child: Text("Takes Space")),  
  ],  
)
```

6 Spacing and Decoration Widgets

Padding

Adds space around a widget.

```
Padding(  
  padding: EdgeInsets.all(10),  
  child: Text("Hello"),  
)
```

SizedBox

Adds fixed width or height spacing.

```
SizedBox(height: 20)
```

Container

Very powerful widget used for:

- Padding
- Margin
- Background color
- Border
- Decoration
- Alignment

```
Container(  
  padding: EdgeInsets.all(10),  
  color: Colors.blue,  
  child: Text("Styled Box"),  
)
```

7 Scrollable Widgets

ListView

Creates a scrollable list.

```
ListView(  
  children: [  
    Text("Item 1"),  
    Text("Item 2"),  
  ],  
)
```

Useful for:

- Product lists
 - Messages
 - Order history
 - Large datasets
-

8 Input and Interaction

TextField

Used to create input fields.

```
TextField(  
  decoration: InputDecoration(  
    labelText: "Enter Name",  
  ),  
)
```

Buttons

Flutter provides multiple button types:

- ElevatedButton
- TextButton
- OutlinedButton

Example:

```
ElevatedButton(  
  onPressed: () {},  
  child: Text("Click Me"),  
)
```

GestureDetector

Used to detect user gestures like:

- Tap
 - Double tap
 - Long press
-

9 State Management Basics

setState()

Used to update UI when data changes.

When `setState()` is called:

- Widget rebuilds
- UI updates
- State is maintained

Example:

```
setState(() {  
  counter++;  
});
```

[?](#) Navigation

Navigator

Used to move between screens.

Go to another page:

```
Navigator.push(  
  context,  
  MaterialPageRoute(builder: (context) => SecondPage()),  
);
```

Go back:

```
Navigator.pop(context);
```

1 Build Method

Every widget contains a `build()` method.

The `build()` method:

- Describes UI
- Returns widgets
- Rebuilds when state changes

Example:

```
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    body: Text("Hello"),  
  );  
}
```



```
} ;  
}
```

12 Hot Reload

Hot Reload allows developers to:

- See changes instantly
- Update UI without restarting app
- Keep app state

This makes development very fast and efficient.

13 Firebase in Flutter

To use Firebase in Flutter, you need:

□ **firebase_core**

It initializes Firebase in your app.

Other useful Firebase packages:

- firebase_auth (Authentication)
 - cloud_firestore (Database)
 - firebase_storage (File storage)
-

14 Compilation

Flutter apps compile to:

- Native ARM code (Android & iOS)
- JavaScript (Web)

This ensures:

- High performance
- Smooth animations
- Fast execution